

Bitácora de Soporte Git & Control de Versiones

DOCUMENTACIÓN DE LIMPIEZA DE REPOSITORIO E IMPLEMENTACIÓN DE VERSIONADO

Proyecto:	Proyecto Hilton (hilton_v2)	Fecha:	17 de junio de 2026
Entorno:	Local (VS Code / Windows Terminal)	Estado Final:	Sincronizado / Despliegue Exitoso

1. Resumen del Problema Inicial

El repositorio local de Git se encontraba reteniendo en el historial de confirmaciones múltiples archivos multimedia pesados (formatos .mp4 y .mkv) ubicados en la carpeta uploads/. Al intentar realizar una carga ordinaria a GitHub, la plataforma rechazaba la transferencia debido a las restricciones de tamaño de archivos, bloqueando las actualizaciones del código fuente (rutas, vistas EJS y estilos).

2. Solución Aplicada: Remoción Física y Limpieza de Git

Para corregir la persistencia de los archivos grandes sin afectar el desarrollo local futuro, se procedió con la remoción directa desde el entorno de desarrollo y la posterior reconfiguración del área de preparación de Git:

- **Eliminación Física:** Se borraron manualmente los archivos multimedia de la carpeta uploads/ desde el explorador de Visual Studio Code.
- **Reajuste del Historial Local:** Se deshizo el commit que mantenía bloqueados los archivos multimedia mediante un reset blando, preservando las modificaciones de código intactas:

```
git reset --soft origin/main
```

3. Configuración e Inclusión del Archivo .gitignore

Con el fin de evitar que futuros archivos cargados en la carpeta de subidas afecten el repositorio, se identificó la raíz del proyecto (HILTON_V2, al mismo nivel que server.js y package.json) y se generó la exclusión correspondiente:

```
# Ubicación: /HILTON_V2/.gitignore
uploads/
```

Git detectó el archivo de exclusión de manera inmediata como un archivo sin seguimiento (untracked file):

```
Untracked files:
  (use "git add <file>..." to include in what will be committed)
.gitignore
```

4. Consolidación de Cambios e Historial de Comandos Ejecutados

Para asegurar que tanto las remociones de los videos como el nuevo archivo de configuración y las actualizaciones de código fueran indexados correctamente, se ejecutó la secuencia final en la terminal:

```
git add .
git commit -m "cambios"
git push origin main
```

La terminal arrojó el siguiente resultado exitoso, confirmando la purga de archivos multimedia en el servidor remoto:

```
git commit -m "cambios"
[main a7d25ae] cambios
22 files changed, 1666 insertions(+), 866 deletions(-)
delete mode 100644 uploads/1780508352633.jpeg
delete mode 100644 uploads/1780508352639.mp4
delete mode 100644 uploads/1781024388463.png
delete mode 100644 uploads/1781024388464.mkv
```

```
git push origin main
Enumerating objects: 47, done.
Counting objects: 100% (47/47), done.
Writing objects: 100% (24/24), 26.65 KiB | 2.42 MiB/s, done.
To https://github.com/mxnadalindev/Proyecto-Hilton.git
690dcd7..a7d25ae  main -> main
```

5. Implementación del Sistema de Versionado (Tags)

Como mejora en las prácticas de despliegue, se implementó el uso de etiquetas anotadas (Semantic Versioning) para congelar el estado del código y documentar los lanzamientos de forma profesional:

```
git tag -a v1.0.0 -m "Primera version estable sin videos"
git push origin --tags
```

✓ Estado Actual del Repositorio

El Proyecto Hilton se encuentra sincronizado en su rama principal (`main`), libre de residuos multimedia pesados y con la etiqueta **v1.0.0** desplegada de forma correcta en GitHub. El entorno local de trabajo reporta un árbol limpio (`working tree clean`).